Akceptesti

FUNKCIONĀLIE TESTI

F1 - CO_2, mitruma, temperatūras un spiediena mērīšana

Testa apraksts:

Apskatīšu, cik ilgi un vai sensors pēc ieslēgšanas uz ekrāna izvada visu sensoru datus (temperatūru, spiedienu, CO2 koncentrāciju un mitrumu)

Ievade/darbība:

Sensoru ar USB-A uz USB-C vadu pieslēdzu pie elektrības avota (datora) un ar hronometru uzņemu laiku līdz pirmajai pilnvērtīgajai izvadei uz sensora ekrāna.

Sagaidāmais rezultāts:

30 sekunžu laikā no ieslēgšanas sensors būs veicis pirmo mērījumu un to izvadīs uz ekrāna.

Patiesais rezultāts:

45 sekunžu laikā sensors uzrādīja pilnvērtīgus mērījumus (CO2 koncentrācija : 1250 PPM, Spiediens : 1028.09 hPa, Temperatūra : 35.52 C, Mitrums : 21.3%

F2 - Sensora savienojums ar datubāzi

Testa apraksts :

Sensors ir savienots ar RPi hotspotu un nosūta mērījumu, tam uzrādoties datubāzē ar konsekventu mērījuma id un laika atzīmi, kurā tas uzņemts.

Ievade/darbība :

Linux terminālī novērot brīdi, kad uzrādās “POST /readings HTTP/1.1” un novērot servera atbildi uz to.

Sagaidāmais rezultāts :

Sensors izveido “POST /readings HTTP/1.1” pieprasījumu, kuru pavada servera atbilde “201 Created”, kā arī pēc tam serveris nosūta “GET /readings HTTP/1.1”, kuru pavada ziņojums “200 OK”.

Patiesais rezultāts:

Sensors veiksmīgi ar “POST” metodi augšupielādē ievāktos datus servera datubāzē, kā arī serveris to bez kļūmēm iegūst ‘GET“ operācijā.

F3 - Jaunāko datu attēlošana vietnē

Testa apraksts:

Pārbaudīšu, vai sensora dati reāllaikā atjaunojas interneta vietnē, salīdzinot 10 uz sensora ekrāna uzrādītos mērījumus tiem, kas tiek izvadīti tīmekļa vietnē.

Ievade/darbība:

Sagaidu jauna mērījuma uzrādīšanos un salīdzinu uz sensora ekrāna redzamās vērtības ar tām, kas uzrādās tīmekļa vietnē kā visnesenākais mērījums. Iepriekšminēto atkārtošu 10 konsekventiem rādījumiem.

Sagaidāmais rezultāts :

Visi mērījumi sakrīt gan sensora ekrānā, gan tīmekļa vietnē.

Patiesais rezultāts :

Visi mērījumi sakrīt, tīmekļa vietnei atsvaidzinoties reāllaikā un uzrādot jaunākos datus, kurus uzņēmis sensors. Taču 4 no 10 CO2 sensora izvadēm uzrādīja vērtību “0”, atšķirībā no visām pārējām datu kopām, no kurām katra uzrādīja loģiskas izvades. Šis liecina par potenciālu CO2 sensora bojājumu.

NEFUNKCIONĀLIE TESTI

N1 - Savienojuma uzticamība

Testa apraksts:

10 minūtes novēroju sensora un servera mijiedarbību, tiem apmainoties ar informāciju, ar mērķi novērot, vai sensors datus ievāc konstanti.

Ievade/darbība:

Palaižu “uvicorn main:app –host 192.168.4.1 –port 8000” komandu un paralēli uzņemu 10 minūšu laika atskaiti, uzskaitot veiktās datu apmaiņas instances starp sensoru un serveri. No iegūtajiem datiem aprēķinam sakritību ar mērķi : (faktiskie / 27 (vidējā vērtība)) × 100%

Sagaidāmais rezultāts:

Sensors darbojas 10 minūtes un nosūta datus reizi 20-25 sekundēs (24-30 paredzēti ieraksti). Process noris ar ≥ 90% precizitāti (≥ 24 ieraksti no 27)

Patiesais rezultāts :

10 minūšu laikā tika ievāktas 27 datu instances, kuras, ievietojot iepriekšminētajā aprēķinā, uzrāda 100% sakritību ar nostādīto mērķi, sensoram veicot datu ievākšanu reizi 20-25 sekundēs.

N2 — Sensora precizitāte

Testa apraksts :

Lai noteiktu sensora precizitāti nepieciešams tā mērījumus salīdzināt ar uzticamu risinājumu, kas šajā gadijumā būs skolas aranet4 sensori. Salīdzinot abu sensoru izvades noteikšu Aeriq sensora precizitāti.

Ievade/darbības :

Novietošu Aeriq sensoru blakus aranet4 sensoram, nodrošinot maksimāli līdzīgus apkārtējos apstākļus. Reizi minūtē nolasīšu abu sensoru rādījumus, tos salīdzinot, tā kopā 10 reizes.

Sagaidāmais rezultāts :

CO_2 novirze ≤ ±40 ppm, temperatūras novirze ≤ ±0.5°C, spiediena novirze ≤ ±0.5 hPa, mitruma novirze ≤ ±2 %

Patiesais rezultāts:

Neviens no mērījumiem neatbilst sagaidāmajos rezultātos noteiktajam diapazonam, taču **spiediens** un **mitrums** ir diezgan līdzīgi. Pirmstam secinājām, ka CO2 sensors ir bojāts, tādēļ no tā sagaidīt precīzus datus nevar. Par temperatūras sensoru gan šādu ziņu nav.

	Temp.	**Spiediens**	**Mitrums**	**CO2**
AERIQ	30 C	1024 hPa	21%	1163ppm
aranet4	24 C	1023 hPa	18%	631ppm

tabula 1, 10 minūšu laikā ievākto datu vidējās vērtības

N3 - Servera noslodze

Testa apraksts:

RPi servera hostotajai tīmekļa vietnei pieslēgšu noteiktu skaitu lietotāju, lai noskaidrotu, vai tā spēj uzturēt vairākus savienojumus vienlaicīgi.

Ievade/darbība:

Ar Linux funkcijām izveidoju skriptu, kas simulēja 100 vienlaicīgus pieprasījumus:

```
for i in {1..100}; do curl -o /dev/null -w "%{time_total}s\n" http://localhost:8000/ & done; wait
```

Sagaidāmais rezultāts :

Tīmekļa vietne spēj uzturēt vairākus pieprasījumus reizē, neuzrādot funkcionalitātes pārtraukumus

Patiesais rezultāts :

Visi 100 pieprasījumi saņēma veiksmīgas “200 OK” atbildes no servera, pierādot tā noturību arī pret lielāku noslodzi. Vidējais atbildes laiks bija 0.032 sekundes

Vienībtesti

Skripti : test_aeriq.c

Es apskatīju kā **bme280_compensate_pressure** (spiediena kalibrācija) un **snprintf** (JSON formatēšana) funkcijas reaģē uz dažādām robežvērtībām, ievietojot nerealistiskus datus, kā arī neievietojot datus vispār.

bme280_compensate_pressure (japievieno skripti, kas palaiz testus)

Spiediena kalibrācijas funkcijai veicu kopā 5 testus (skat. attēls 1), kurus referencēju pret normālu mērījumu:

1. Temperatūra = 0
2. Spiediens = 0
3. Spiediena vērtība ir negatīva
4. Spiediena vērtība ir maksimāla
5. Dati, kas nepieciešami kalibrācijai, netiek ielādēti

Pēc šiem testiem varu secināt, ka programma pilnībā uzticas tai padotajiem datiem un tik un tā tos apstrādā, pat ja datiir maldīgi/neiespējami, uzrādot sķietami reālu, bet praktiski kļūdainu vērtību.

Vienīgais mehānisms, kas nostrādāja, bija gadījumā, kad funkcija nesaņēma <u>nekādas</u> vērtības, kurā gadījumā tika atgriests mērījums 0.0

```
Normal input   (adc_P=415148, t_fine=128422):  1006.53 hPa
t_fine = 0     (temperature not computed yet):  968.21 hPa
adc_P = 0      (no pressure reading):           1731.99 hPa
adc_P = -1     (negative raw value):            1732.00 hPa
adc_P = MAX    (2147483647):                    232684704.00 hPa
Calib = all 0  (calibration never loaded):      0.00 hPa
```

Attēls 1, bme280_compensate_pressure testa ievades un rezultāti

snprintf

Līdzīgi kā spiediena kalibrācijas testos, arī JSON formatēšanas funkcijai izpildīju 5 testus (skat. attēls 2) testējot robežgadījumus, un salīdzinoot tos ar normālu ievadi:

1. Visas nosūtītās vērtības ir 0
2. Tiek nosūtīti negatīvas temperatūras dati

3. Tiek nosūtīta neiespējama CO2 vērtība
4. Tiek nosūtīta neiespējama mitruma vērtība
5. Visos datu lauciņos ir absurdi ekstrēmas vērtības

Visas iepriekšminētās ievades izgāja cauri un tika nosūtītas uz displeju, kas nozīmē, ka arī šī funkcija nepārbauda ievadītās vērtības.

```
Normal input:
  {"temperature":22.5,"humidity":48.3,"co2":850,"pressure":1013.25,"bme_temperature":23.1,"bme_humidity":47.0}

All zeros:
  {"temperature":0.0,"humidity":0.0,"co2":0,"pressure":0.00,"bme_temperature":0.0,"bme_humidity":0.0}

Negative temperature (-40.0):
  {"temperature":-40.0,"humidity":48.3,"co2":850,"pressure":1013.25,"bme_temperature":-40.0,"bme_humidity":47.0}

CO2 = 65535 ppm (max uint16):
  {"temperature":22.5,"humidity":48.3,"co2":65535,"pressure":1013.25,"bme_temperature":23.1,"bme_humidity":47.0}

Humidity = 150% (over 100):
  {"temperature":22.5,"humidity":150.0,"co2":850,"pressure":1013.25,"bme_temperature":23.1,"bme_humidity":150.0}

All extreme floats:
  {"temperature":99999.9,"humidity":99999.9,"co2":65535,"pressure":99999.99,"bme_temperature":99999.9,"bme_humid
ity":99999.9}
```

2026. gada 13.marta 11:39

```
200|1773394745.80347|2026-03-13 11:39:05|37.82|27.54|1310.0|1026.78|40.98|17.91
199|1773394720.73728|2026-03-13 11:38:40|37.79|27.58|1314.0|1026.83|40.97|17.97
198|1773394700.69378|2026-03-13 11:38:20|37.79|27.68|1319.0|1026.84|40.95|18.01
197|1773394675.64617|2026-03-13 11:37:55|37.75|27.74|1322.0|1026.77|40.94|17.78
196|1773394655.60525|2026-03-13 11:37:35|37.73|27.75|0.0|1026.81|40.91|18.1
195|1773394635.56665|2026-03-13 11:37:15|37.72|27.77|0.0|1026.83|40.91|18.12
194|1773394610.53193|2026-03-13 11:36:50|37.72|27.82|1342.0|1026.86|40.89|18.19
193|1773394590.47475|2026-03-13 11:36:30|37.69|27.87|0.0|1026.84|40.86|18.38
192|1773394565.42191|2026-03-13 11:36:05|37.66|27.9|1358.0|1026.84|40.84|18.31
191|1773394545.39788|2026-03-13 11:35:45|37.64|27.91|1373.0|1026.81|40.82|18.36
190|1773394520.33652|2026-03-13 11:35:20|37.63|27.97|1382.0|1026.84|40.79|18.44
189|1773394495.28152|2026-03-13 11:34:55|37.61|28.01|1398.0|1026.86|40.77|18.16
```

Attēls 3, 2026. gada 15. aprīļa 14:57 ievāktie dati

Integrācijas testi

Skripti : integration_tests.c

Lai izvērtētu programmas funkciju savstarpējo mijiedarbību veicu 2 integrācijas testus, kas apraksta, kā dažādas funkcijas ir cita no citas atkarīgas :

1. Temperatūras -> spiediena mērījumu kalibrācija
2. Datu apstrāde -> JSON objekts uz RaspberyPi (RPi) serveri

Temperatūras - spiediena mērījumu kalibrācija

Kā vienā no iepriekš veiktajiem akcepttestiem novērojām, spiediena mērījuma kalibrācijas gala dati mainījās, ja iepriekš netika veikts temperatūras mērījums. Lai apstiprinātu šo funkciju savstarpējo atkarību, es pārbaudīju kā mainās spiediena vērtība atkarībā no temperatūras mērījuma klātbūtnes un kā izmantoju standarta izvadi kā atsauces vērtību (skat. 4. attēls) .

```
Step 1 - Pressure WITHOUT temperature first:
  t_fine = 0
  pressure = 968.21 hPa

Step 2 - Temperature computed first:
  adc_T = 519888
  temperature = 25.08 C
  t_fine = 128422

Step 3 - Pressure WITH temperature first:
  adc_P = 415148
  pressure = 1006.53 hPa

Difference: 38.32 hPa
```

4. attēls, spiediena atkarība no temperatūras mērījuma klātbūtnes

Kā 4. attēlā redzams, programma izvada spiediena mērījumu **pat ja temperatūras mērījuma nav**. Tas nozīmē, ka tiek izvadīti maldinoši dati, kas bez konteksta šķiet patiesi. Taču varam secināt, ka abas funkcijas darbojas sinhroni, vienai ar otru apmainoties datiem un tos apstrādājot.

Datu apstrāde -> JSON objekts uz RaspberyPi (RPi) serveri

Apstrādātie dati tiek augšupielādēti uz serveri un izvadīti uz sensora displeja ar JSON objektu palīdzību, kas ir iemesls kāpēc tā mijiedarbība ar datu apstrādes bloku ir vitāli svarīga. Lai to pārbaudītu, es apskatīju vai datu apstrādē iegūtās vērtības veiksmīgi tiek ievietotas JSON objektā tālākai apstrādei (skat. 5. & 6. attēls)

```
Step 1 - Raw ADC values:
  adc_T = 519888
  adc_P = 415148

Step 2 - After compensation:
  bme_t = 25.08 C
  bme_p = 1006.53 hPa

Step 3 - JSON sent to RPi server:
  {"temperature":22.5,"humidity":48.3,"co2":850,"pressure":1006.53,"bme_temperature":25.1,"bme_humidity":47.0}
```

5. attēls, apstrādāto datu ievietošana JSON objektā tālākai apstrādei

```
Step 1 - Raw ADC values:
  adc_T = -796917760
  adc_P = -973078528

Step 2 - After compensation:
  bme_t = -7479247.50 C
  bme_p = 78.62 hPa

Step 3 - JSON sent to RPi server:
  {"temperature":22.5,"humidity":48.3,"co2":850,"pressure":78.62,"bme_temperature":-7479247.5,"bme_humidity":47.0}
```

6. attēls, apstrādāto datu ievietošana JSON objektā tālākai apstrādei

Apskatot iepriekš ievietotos attēlus varu secināt, ka datu apstrāde veiksmīgi komunicē ar JSON objektu izveides funkciju, apkopojot iegūtos rādījumus un nosūtot tos tālāk RPi serverim.

Programmas novērtējums(2-3 teik.)

Vispārīgi, programmas kods uzrakstīts saprotami, ļoti palīdz arī komplementārie komentāri, kas pievienoti katras sadaļas sākumā. Kaut arī iekārtas mērījumi ir kļūdaini, tās pamata funkcionalitāte strādā un izvada loģiskus mērījumus. Vienīgais apsvērums, ko es gribētu lai ņem vērā ir datu validācija pirms to nosūtīšanas uz serveri/sensora ekrānu, kaut kāda veida selektīvu atlasi, kas atpazīst kļūdainus datus un pārtrauc to tālāku nosūtīšanu.